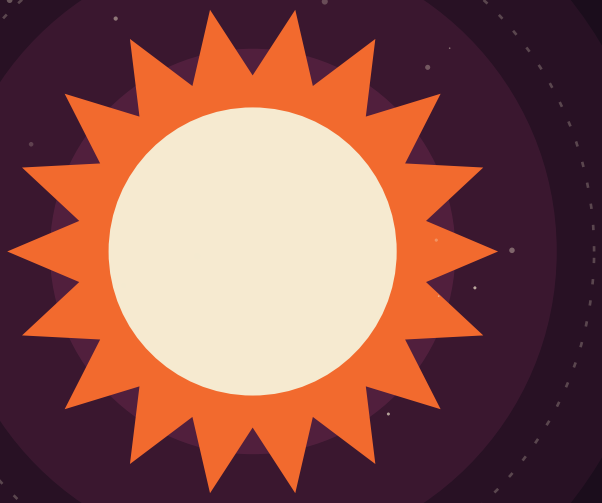
ADITYA-L1 SPACE WEATHER MONITORING PIPELINE



FlareWatch

Solar flare nowcast, 15-minute forecast, continual learning and live dashboard

Project report: models, data, retraining, UI/UX, results and next steps

Status as of 29 October 2026 · repository branch main

Data: NOAA GOES-19 XRS (NCEI science archive + SWPC real-time feed), NOAA flare list; ISRO Aditya-L1 SoLEXS / HEL1OS (PRADAN).

Contents

Contents 2

1. Summary 3

2. System overview 4

3. Where we started: the original prototype 5

4. Data 6

5. Model 1 — the nowcast (l1_nowcast) 7

6. Models 2 and 3 — the 15-minute forecast (l1_forecast) 10

7. Continual learning: daily retraining (continual/) 13

8. FlareWatch dashboard: architecture 15

9. UI / UX design 17

10. Testing, code layout and history 21

11. Limitations and next steps 23

Glossary of scores 23

How to read this report. Sections 1–2 give the big picture. Sections 3–6 explain each model and the data it uses, with held-out results. Section 7 covers how the system retrains itself every day. Sections 8–9 cover the live dashboard and its UI/UX design. Section 10 lists tests, code layout and commits, and Section 11 the limitations and next steps. A glossary of the scores (POD, FAR, CSI and so on) is at the end.

1. Summary

This project watches the Sun's soft X-ray brightness and answers two questions in real time: **is a solar flare happening right now, and how big is it?** (the *nowcast*), and **will a flare start in the next 15 minutes?** (the *forecast*). It was built for ISRO's Aditya-L1 instruments (SoLEXS soft X-ray and HEL1OS hard X-ray), and it is validated on NOAA's GOES-19 XRS, which has an independent official flare list to score against.

Around the two models we built a **continual-learning loop**: every day's data is stored, both forecast models are retrained on the whole history, and a new version replaces the running one only if it does better on recent days that neither model has seen. Everything is shown in **FlareWatch**, a live web dashboard with an alarm board, charts, an alarm log, model cards and day, dawn and night themes that follow the viewer's own clock.

Headline results (held-out data, never used for tuning)

Component	Test period	Result
Nowcast (flare detection)	1 May – 27 Oct 2026, 936 NOAA flares ≥ C1	Precision 0.91, recall 0.91, F1 0.91. M-class 97/99 caught, X-class 3/3. Class letter matches NOAA 99.9 %. Alarm a median 3 min **before** the peak.
Forecast ≥ C1 in 15 min	1 Jul – 27 Oct 2026, 542 flares	Catches 66 % (POD 0.66), 36 % of alarms false, CSI 0.47, 1.8 false alarms/day. Halves the false alarms of a simple rise rule.
Forecast ≥ M1 in 15 min	1 Jul – 27 Oct 2026, 79 flares	POD 0.44, CSI 0.16, ROC AUC 0.85. Median warning 13 min before the nowcast alarm; 57 % of hits warn before the flare physically starts.
Continual learning	Replay test + first live run 29 Oct	Replay: C1 challenger promoted (log loss 0.129 vs 0.142); M1 challenger correctly rejected. Live: both kept (not enough unseen days yet).
Dashboard	Live on Vercel and locally	Full snapshot (fetch → nowcast → forecast → JSON) in ~3 s, 71 KB gzipped, refreshed every 30 s.

What changed from the starting point

- The original notebook (`model.ipynb`) claimed **100 % detection, 10.7 % false alarms and an 11.3-minute warning**. We found seven methodological problems (Section 3) that made those numbers invalid, and replaced them with honest, held-out ones.
- The flare labels it learned from came from a single global threshold (mean + 5σ) that caught only 3 % of real flares. We built a proper streaming nowcast that catches 91 %.
- We added a 15-minute forecaster with 40 causal features, two gradient-boosted models, calibration and alert rules.
- We added daily retraining with a champion/challenger safety gate, a live dashboard on public real-time data, and a full UI redesign with themes.

2. System overview

The pipeline is built so that **the same code runs on a live stream and on years of archived data**. Both engines take one sample at a time and keep only a small amount of state (a rolling window), so a back-test is simply the live loop run faster. Batch versions give identical numbers and are used by the dashboard.

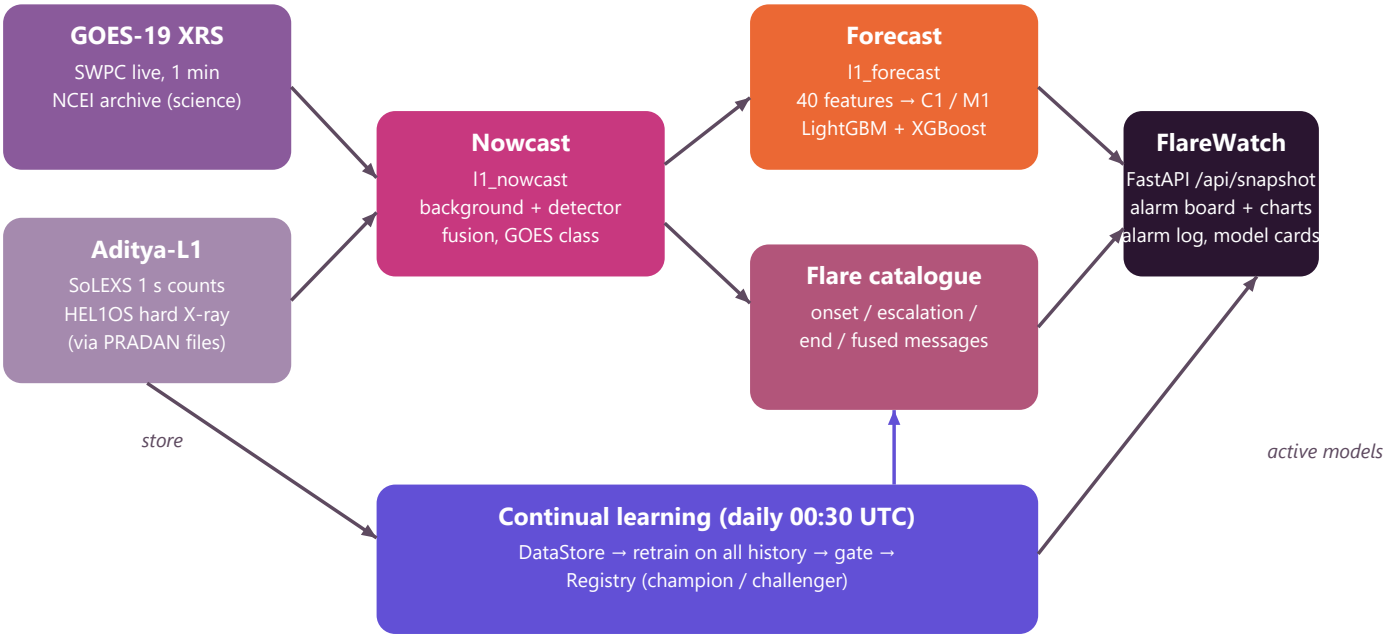


Figure 2.1 — Data flows left to right. The nowcast turns raw X-ray flux into flare events; the forecast uses the flux and the nowcast's state to predict new flares; the dashboard shows both. Every night the continual-learning job retrains the forecast models and swaps them in only if they pass the gate.

Components (all in one repository root)

Folder	What it is	Main entry points
l1_nowcast/	Python package `l1_nowcast` (notebook in notebooks/). Streaming flare detector, background estimator, two-channel fusion, data readers (GOES NCEI, SWPC live, SoLEXS, HEL1OS, directory watch), validation against NOAA.	NowcastEngine, run_batch, tune_channel, GOESLiveSource
l1_forecast/	Package `l1_forecast`. Features, labels, ensemble model, walk-forward training, alert rules, streaming forecast engine. Saved models in models/{C1, M1, C1_transfer}.	build_features, FlareForecaster, walk_forward, AlertManager, ForecastEngine
continual/	Daily store-and-retrain loop: DataStore, Registry, train / evaluate / promote. CLI: seed, ingest, daily, status.	python -m continual daily
api/, public/, flaremon/	FlareWatch: api/index.py is the FastAPI (Vercel) function, public/ the static page, flaremon/ the snapshot builder. serve.py runs both locally, with an optional daily retrain thread.	python serve.py --retrain-daily
.github/workflows/	daily-retrain.yml: GitHub Actions cron (00:30 UTC) that runs the daily cycle and commits the store and registry.	—

3. Where we started: the original prototype

The repository began with a forecasting notebook (`model.ipynb` / `model.pdf`), two plots and a README describing a "Stage 2 deployment". Related Team Phoenix scripts used a rolling-median flare detector. The notebook reported excellent numbers on 25 test flares:

Metric (as claimed)	TPR	FAR	CSI	Mean warning
model.ipynb	100 %	10.7 %	0.89	11.3 min

When we checked the method, the numbers did not hold up. Each problem below inflates the score; together they make it meaningless:

#	Problem in the original	Why it matters	How it is fixed now
1	Labels from whole-dataset statistics (mean + 5σ over all data, including the test period)	Future information leaks into training; and the rule catches only 3 % of NOAA flares	Labels come from the causal streaming nowcast, which only sees the past
2	One flare counted several times	Inflates the number of hits	Event-based scoring: one flare = one event, matched to NOAA
3	Lead time measured to the peak	Peak comes minutes after the start, so warning looks longer	Lead measured to the nowcast alarm and to NOAA's start time
4	Rows during a flare were scored	Predicting "flare" while one is visibly in progress is trivial	Only rows not already in FLARE state are scored
5	LightGBM 0/1 predict() averaged with probabilities	Mixes hard labels with probabilities; breaks the ensemble	Probabilities averaged, then isotonic calibration
6	Row-based rolling windows across data gaps	A "30-row" window could span hours or days	Time-based, gap-aware windows with validity fractions
7	Only 25 test events, single split	Very large uncertainty; luck dominates	Walk-forward monthly retraining over 4 months, 542 C1 and 79 M1 flares

Baselines kept for comparison (nowcast, May–Oct 2026)

Detector	Precision	Recall ≥ C1	F1	M POD	X POD
Global mean + 5σ (original labelling rule)	0.96	0.03	0.05	0.23	1.00
Team Phoenix-style rolling median	0.75	0.73	0.74	0.86	1.00
This nowcast engine	**0.91**	**0.91**	**0.91**	**0.98**	**1.00**

4. Data

Source	What	Cadence / span	Used for
NOAA NCEI GOES-19 XRS L2 (science archive)	Calibrated soft X-ray flux: XRS-B 1–8 Å and XRS-A 0.5–4 Å, with quality flags	1 min; 1 Jan – 27 Oct 2026 (~100 MB)	Tuning and validating the nowcast; training and testing the forecast; permanent history of the store
NOAA SWPC real-time JSON (xrays-3-day / 7-day)	Same two channels from the operational feed, public, no login	1 min, last 3 or 7 days, updated each minute	Live dashboard; daily ingest into the store
NOAA GOES flare list	Official start / peak / end time and class of each flare	Event list	Ground truth for scoring the nowcast (936 flares ≥ C1 in May–Oct)
ISRO Aditya-L1 SoLEXS	Soft X-ray counts (2–22 keV)	1 s, files via PRADAN	Target instrument. Reader + counts→flux calibration implemented and tested; not yet scored (needs user's PRADAN data)
ISRO Aditya-L1 HEL1OS	Hard X-ray counts	1 s, files via PRADAN	Secondary channel for fusion on Aditya-L1

Why GOES and not Aditya-L1 for validation? ISRO publishes Aditya-L1 data as files on PRADAN, which needs a personal login, and there is no public real-time Aditya-L1 API. GOES-19 has the same kind of soft X-ray measurement, a public 1-minute live feed and an independent flare list, so it is the fair place to measure skill. The Aditya-L1 path uses the same engines with instrument-specific settings and a transfer model (C1_transfer) that uses only features both instruments share.

Data quality handling

- Samples with quality flags, NaN or non-positive flux are dropped; a gap longer than the allowed limit resets the detector to QUIET with a data-gap message.
- All features are time-based and gap-aware: each rolling window reports the fraction of valid minutes (valid_frac60) so the model knows when data are thin.
- In the store, the NCEI science archive wins over the SWPC operational feed wherever both cover the same minute.
- GOES flare class comes from peak flux: A < 10^{-7}, B < 10^{-6}, C < 10^{-5}, M < 10^{-4} W/m², X above.

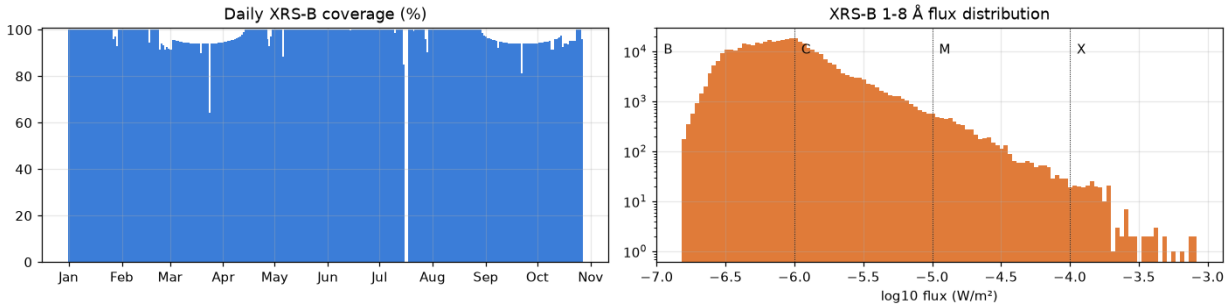
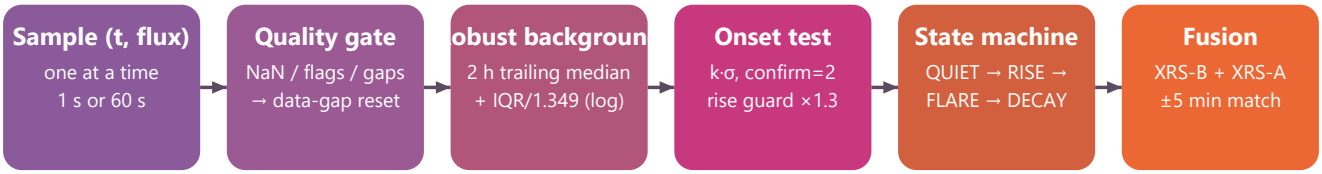


Figure 4.1 — Data coverage and the distribution of XRS-B flux in the 2026 archive.

5. Model 1 — the nowcast (l1_nowcast)

The nowcast is not a machine-learning model; it is a streaming statistical detector, tuned on data. It reads one sample at a time and decides whether the Sun is quiet or flaring, when the flare started, how big it got, and when it ended.



Outputs: onset · escalation (class grows) · end (NOAA half-way rule) · fused (BOTH / PRIMARY_ONLY / SECONDARY_ONLY)

Figure 5.1 — The nowcast processing chain for each new sample.

How it works

- **Background.** A trailing 2-hour window of *quiet* samples only. The level is the median of log-flux and the spread is IQR / 1.349 (a robust standard deviation). Flare samples are not admitted, so a long flare cannot raise its own background. If no quiet sample arrives for 6 hours, the background re-seeds.
- **Onset.** A flare starts when log-flux exceeds background + k·σ for 2 consecutive samples *and* the flux is at least 1.3× its minimum over the last 5 samples (rise guard, stops slow drifts from triggering). A NOAA-style sustained-rise rule can also start a flare.
- **Escalation.** While a flare grows, a message is sent each time its class goes up (e.g. C5 → M1).
- **End.** NOAA's half-way rule: the flare ends when flux falls half-way back from the peak to the pre-flare level. A clear re-rise during decay splits it into a new flare.
- **Fusion.** XRS-B and XRS-A (or SoLEXS and HEL1OS) run separate detectors. Events within ±5 minutes are fused and labelled BOTH, PRIMARY_ONLY or SECONDARY_ONLY, which gives a confidence level.
- **Aditya-L1.** SoLEXS counts are converted to an equivalent GOES flux with a fitted calibration so that the flare class is comparable.

Tuned settings (tuned on Jan–Apr 2026 only)

Channel	k_sigma	min_ratio (rise guard)	Confirm	Background window	Method
XRS-B (1–8 Å, primary)	1.5	1.3	2 samples	7200 s	log_robust
XRS-A (0.5–4 Å, secondary)	3.0	1.5	2 samples	7200 s	log_robust

Results on held-out data (1 May – 27 Oct 2026, 936 NOAA flares ≥ C1)

Score	Value	Meaning
Precision	0.91	91 % of detected flares are real NOAA flares
Recall (POD) ≥ C1	0.91	91 % of NOAA flares are detected
F1	0.91	Balance of the two
M-class POD	0.98 (97 / 99)	Almost every major flare is caught
X-class POD	1.00 (3 / 3)	All the largest flares caught
Class letter agreement	99.9 %	Our class matches NOAA's for matched flares
Alert timing	median 6 min after NOAA start, 3 min before peak	NOAA's start is set afterwards; we alert live while the flare is still rising

Score	Value	Meaning
Speed	~25,000 samples/s	A year of 1-minute data in about 20 s on a laptop

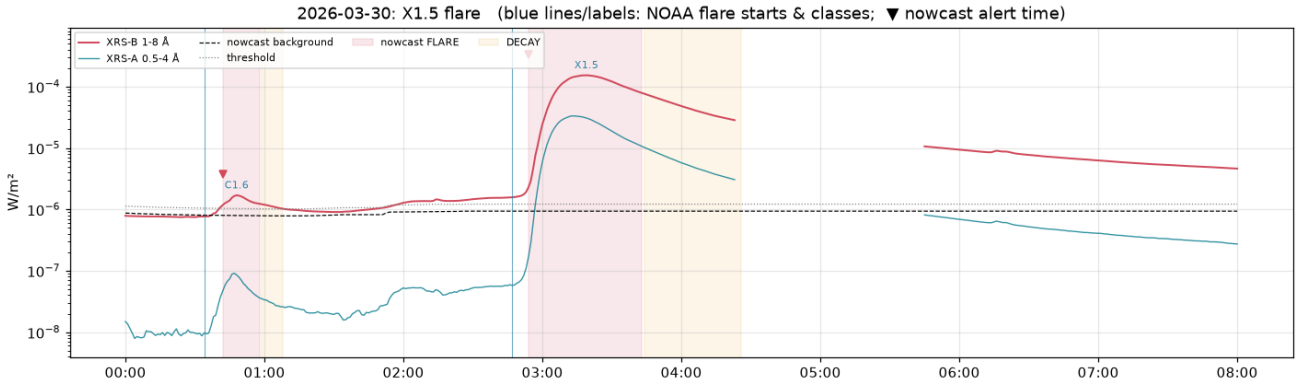


Figure 5.2 — The X1.5 flare of 30 March 2026: XRS-B (red) and XRS-A (teal), background (dashed), FLARE (pink) and DECAY (yellow) states, NOAA start times (grey lines) and the nowcast alert time (triangles).

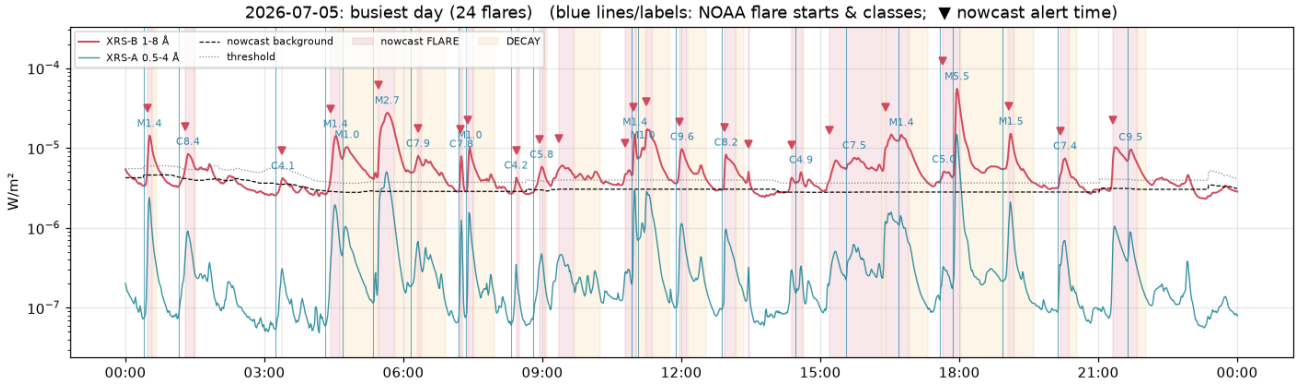


Figure 5.3 — The busiest day in the test period: many overlapping flares are separated by the re-rise rule.

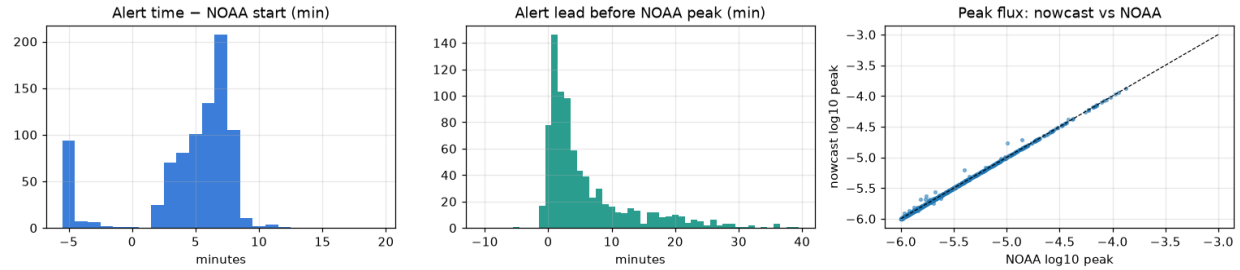


Figure 5.4 — Timing of nowcast alerts relative to NOAA start and peak, and peak-flux agreement.

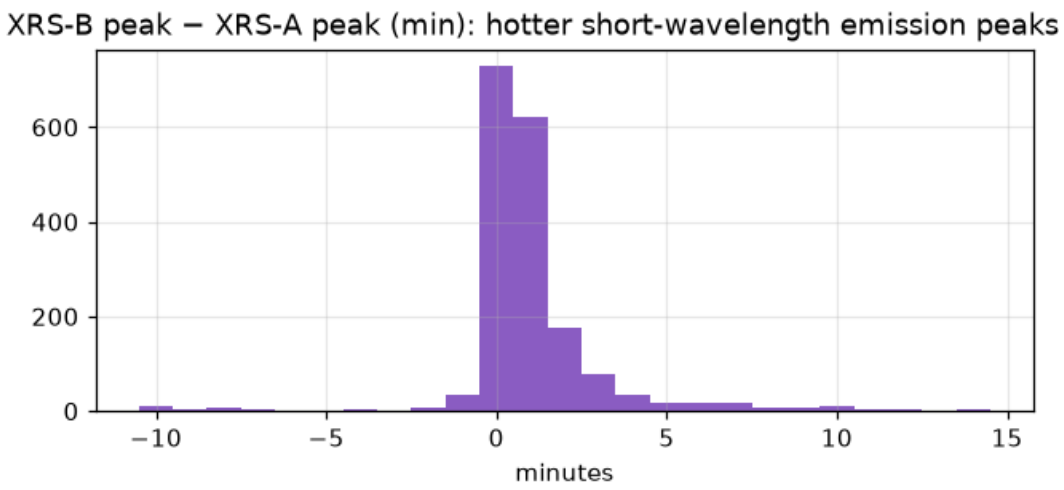


Figure 5.5 — Peak lag between the two channels, which sets the ±5 min fusion window.

6. Models 2 and 3 — the 15-minute forecast (l1_forecast)

The forecast answers: **will the nowcast raise an alarm for a flare of at least class C1 (model C1) or M1 (model M1) within the next 15 minutes?** It is scored only on minutes when no flare is already in progress, so it cannot score points by "predicting" a flare that is already visible.

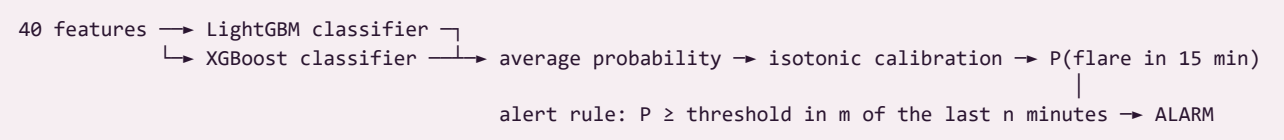
Inputs: 40 causal features

Every feature uses only past data, is computed in real time from the stream, and is aware of data gaps. "p" = primary channel (XRS-B / SoLEXS), "s" = secondary (XRS-A / HEL1OS), "exc" = log excess over the nowcast background.

Group	Features	What they capture
Current excess	p_exc, p_level, b_level, p_exc_d3, s_exc	How far above background the Sun is now, and the background level itself
Rate of change	p_d1, p_d3, p_d5, p_d10, p_d30, s_d1, s_d3, s_d5, s_d10	Rising flux over 1–30 minutes: the strongest short-term signal
Recent maxima	p_exc_max30, …max60, …max180, …max360, …max1440, p_max1440	Activity over the last 30 min to 24 h
Variability	p_vol10, p_vol30, p_vol60	Flickering, which often precedes flares
Rise shape	p_above_min10, p_above_min30, p_rise_run	How far and how long the flux has been climbing
Spectral hardness	tr, tr_d1, tr_d3, tr_d5, tr_d10, tr_dev120	Ratio XRS-A / XRS-B (temperature proxy); heating before a flare
Flare history	in_decay, onsets_6h, onsets_24h, min_since_onset, min_since_end	Active regions keep flaring; time since the last event
Data quality	valid_frac60	Share of valid minutes in the last hour

C1_transfer, for Aditya-L1, drops the features whose meaning differs between instruments, so it can be applied to SoLEXS/HEL1OS before Aditya-L1 has its own training history.

Model structure



- **Two gradient-boosted tree models** (LightGBM 4.7 and XGBoost 3.4) are trained on the same rows and their probabilities averaged. Trees handle the mix of scales and non-linear thresholds in flux data well, and averaging two libraries reduces variance.
- **Early stopping** on the last 15 % of training rows (by time) picks the number of trees without touching the test period.
- **Isotonic calibration** maps raw scores to honest probabilities, so "30 %" really means about 3 in 10.
- **Alert rule** turns probability into an alarm with a threshold and an m-of-n persistence filter to stop flicker: C1 alarms at P ≥ 0.428 (1 of 1 minute); M1 at P ≥ 0.0735 in 3 of the last 5 minutes. M1's threshold is low because M flares are rare (base rate well under 1 %).
- **Two inference engines.** Training uses LightGBM and XGBoost. For prediction, the saved trees (`lgb_model.txt`, `xgb_model.json`) can also be evaluated in pure numpy (`l1_forecast/treeinfer.py`). The loader uses the native libraries when available and numpy otherwise; results are identical (LightGBM exactly, XGBoost within 4×10^{-7}), checked by a test on all three models.
- **Walk-forward training.** Development on Mar–Jun; test Jul 1 – Oct 27. Each test month is predicted by a model retrained on all data before it, with a 1-day embargo between training and test, exactly as it would run live.

Results (1 Jul – 27 Oct 2026, walk-forward)

Model	Flares	POD	FAR	CSI	False alarms / day	ROC AUC	BSS	Median warning
≥ C1	542	0.66	0.36	0.47	1.8	0.79	+0.12	1 min
≥ M1	79	0.44	0.78	0.16	1.2	0.85	+0.05	13 min
≥ C1 transfer	542	0.69	0.43	0.44	2.5	0.77	–	1 min
Baseline: rise rule (no ML)	542	0.65	0.55	0.36	3.7	–	–	1 min

- **C1:** same detection rate as the rule-based baseline but **half the false alarms** (1.8 vs 3.7 per day). Most of its warning is the first minutes of the flare's own rise, so it is best read as a very early confirmation.
- **M1:** shows a genuine precursor signal — the median warning is 13 minutes before the nowcast alarm (4 minutes before NOAA's start), and 57 % of hits warn before the flare physically begins. Because M flares are rare it has many false alarms, so the dashboard presents it as a graded risk ("Major watch").
- Skill varies by month: C1 POD fell to 0.24 in the quiet September, when there were few flares to catch.

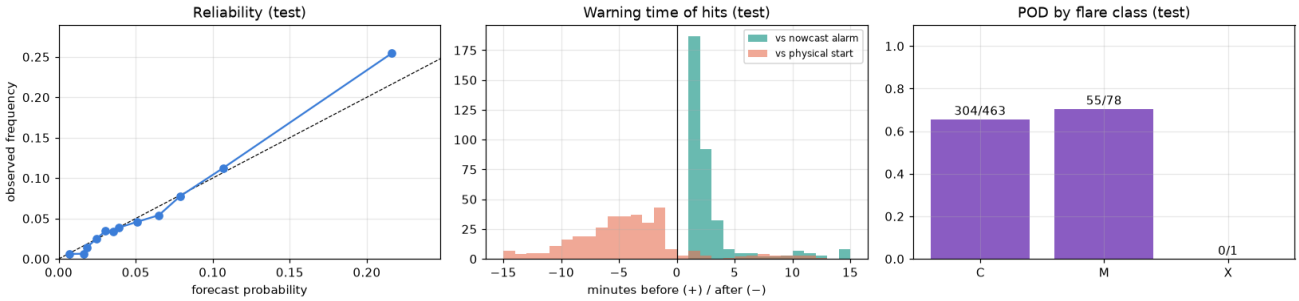


Figure 6.1 — C1 model diagnostics: reliability (calibration), ROC and lead-time distribution.

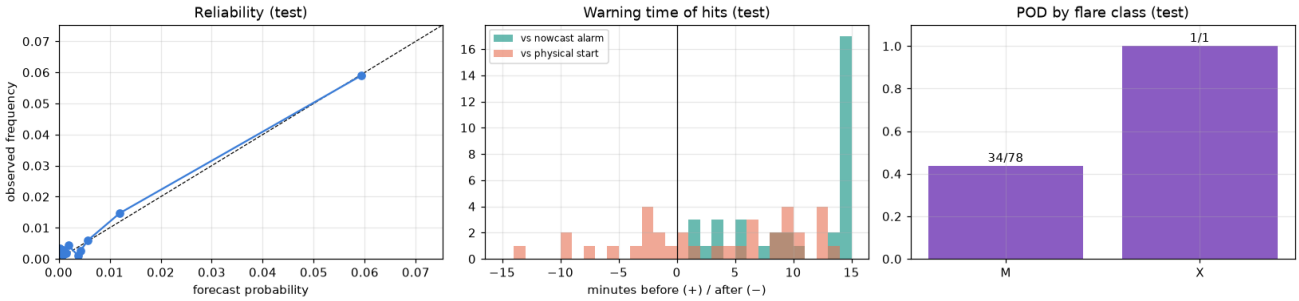


Figure 6.2 — M1 model diagnostics.

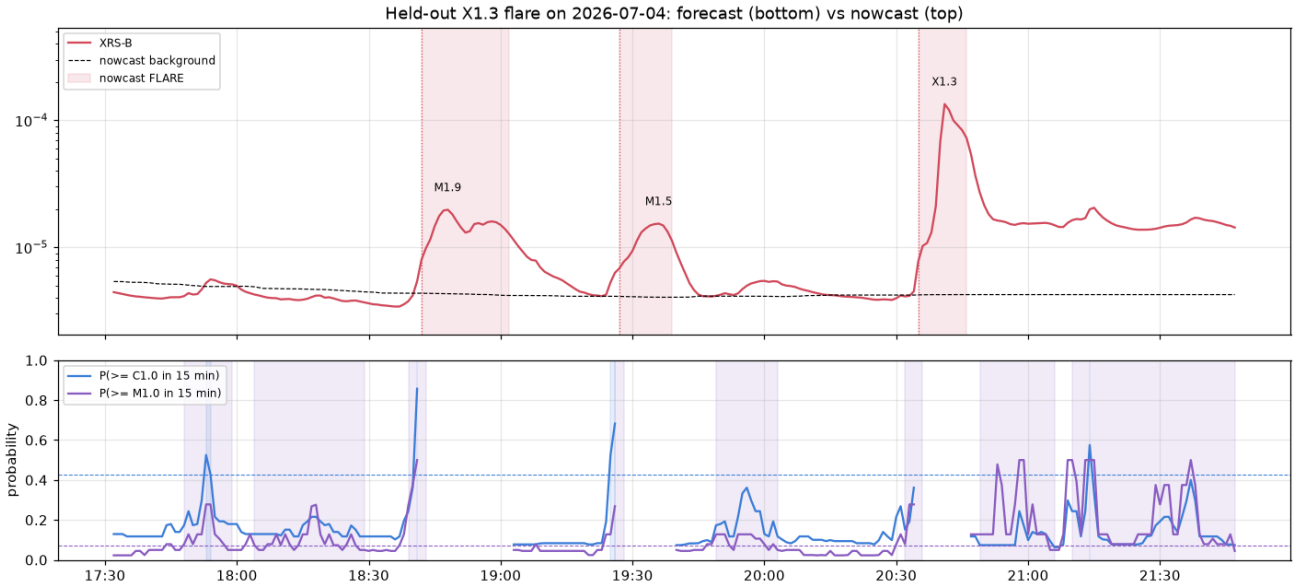


Figure 6.3 — Held-out 4 July 2026: M1.9, M1.5 and X1.3 flares. Top: flux and nowcast FLARE periods. Bottom: forecast probabilities (blue C1, purple M1), alarm thresholds (dashed) and alarm periods (shaded). Gaps are minutes already in a flare.

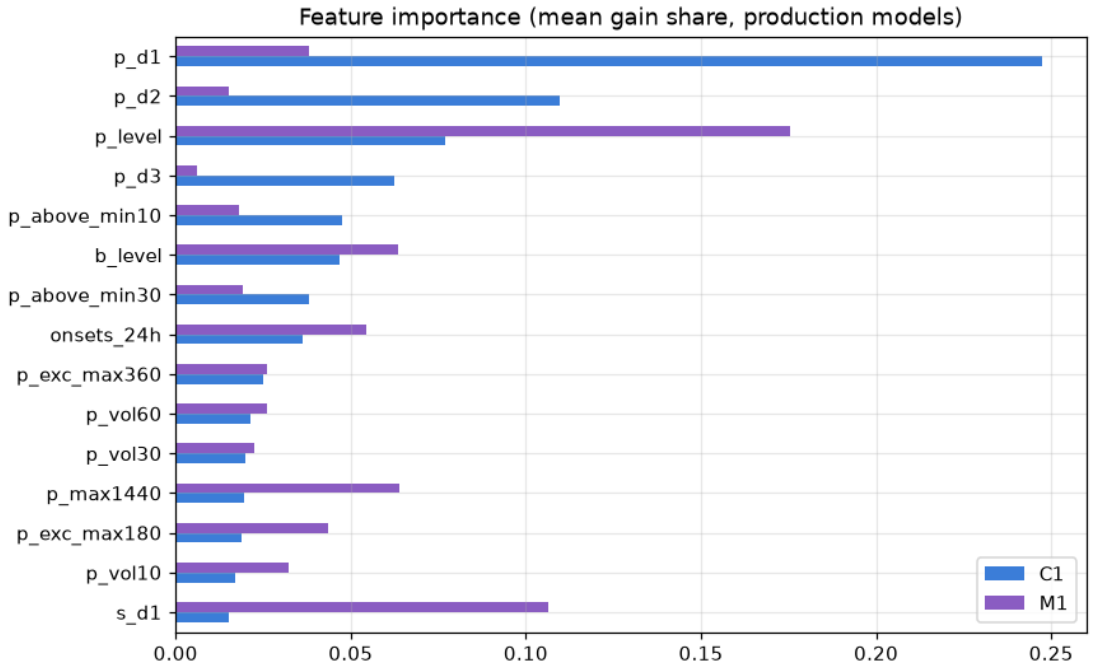


Figure 6.4 — Feature importance: recent rise rates and current excess dominate, followed by hardness ratio and flare history.

7. Continual learning: daily retraining (continual/)

The deployed forecaster keeps learning. Each day it stores the new data, retrains both models, tests the new version against the running one, and only switches if the new one is at least as good. This protects against a bad day of data or a quiet period making the model worse.

Why train on all history, not just the last few days

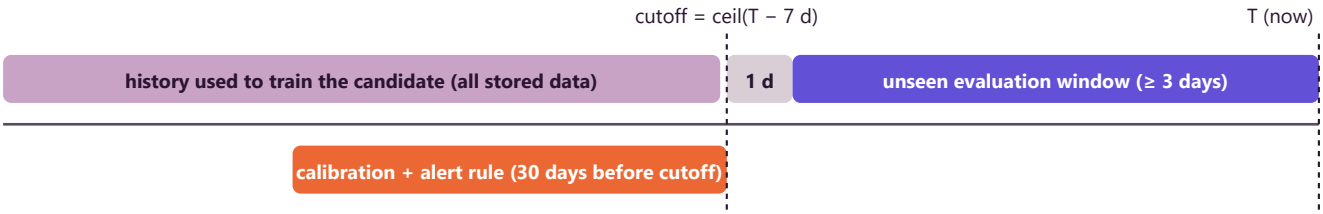
The original idea was to keep 5–10 days of data and train each day on the last day. We kept the 10-day rolling store but train on the **full history**, because a few days contain very few flares (often zero M flares), so a model trained only on them would forget what big flares look like. The 10-day cache is the raw live buffer; the permanent history is what the model learns from.

Storage (DataStore)

Location	Contents	Retention
continual/store/history/xrs_YYYY-MM.parquet	Merged 1-minute XRS-B and XRS-A, one file per month. NCEI archive wins over SWPC where both exist.	Permanent
continual/store/live/YYYY-MM-DD.parquet	Raw daily pulls from the SWPC 7-day feed	Rolling 10 days

At the first live run (29 Oct 2026) the store held 420,755 minutes from 1 Jan to 28 Oct 2026, about 8 MB.

The daily cycle



Promote the challenger only if: log loss ≤ 1.005 × champion AND CSI ≥ champion − 0.05 on the same unseen days.

Evaluation starts at max(cutoff, champion's train_end) + 1-day embargo, so neither model has seen those days.

Figure 7.1 — Time windows used each day. T is the run time.

- **1. Ingest.** Pull the SWPC 7-day feed, save today's raw file, merge into the monthly history, prune live files older than 10 days.
- **2. Build data.** Run the nowcast over the full history to get labels and states, then compute the 40 features.
- **3. Train the candidate** on all history before the cutoff (cutoff = T − 7 days, rounded up to midnight).
- **4. Calibrate** and fit the alert rule on the 30 days before the cutoff, using a model that had not seen those 30 days.
- **5. Evaluate both** the candidate and the current champion on the same unseen window: from max(cutoff, champion's training end) + 1-day embargo up to T. At least 3 unseen days are required, otherwise the champion is kept.
- **6. Decide.** Promote if log loss ≤ champion × 1.005 *and* CSI ≥ champion − 0.05. Log loss measures probability quality across every minute; the CSI guard stops a model that is smoother but misses flares.
- **7. Record.** Save the version, update active.json, append the decision to log.jsonl, keep the active version plus the two newest.

Registry

```
continual/registry/
├── C1/20261028/        model files + meta (train_end, metrics)
```

```
├── M1/20261028/
├── active.json          which version each model uses now
└── log.jsonl            one line per daily decision (shown on the dashboard)
```

The base champion is the model in models/ (trained through 27 Oct). The dashboard always loads whatever active.json points to, so a promotion takes effect on the next refresh without a restart.

Scheduling

- **Local:** `python serve.py --retrain-daily` starts a background thread that runs `python -m continual daily` at 00:30 UTC (and at start-up if today's run is missing).
- **Cloud:** `.github/workflows/daily-retrain.yml` runs the same command on a GitHub Actions cron (30 0 * * *) and commits the updated store and registry.
- Manual commands: `python -m continual seed | ingest | daily | status`.

Results so far

Run	Model	Champion	Challenger	Decision
Replay test (simulated day)	C1	log loss 0.142, 0 of 16 flares	log loss 0.129, 2 of 16 flares	**Promoted**
Replay test (simulated day)	M1	log loss 0.0013	log loss 0.0037	Rejected (worse)
First live run, 29 Oct 05:02	C1, M1	20261028	—	Kept: no unseen days yet

The first fair comparison is expected around 1 November 2026, when three unseen days have accumulated after the champion's training end.

8. FlareWatch dashboard: architecture

```
NOAA SWPC xrays-3-day.json → run_batch (nowcast) → build_features → C1 / M1 (active versions) → JSON → browser
```

- **Stateless snapshot** (`flaremon/snapshot.py`): each call fetches the last 3 days, runs the nowcast and both models in batch (identical to streaming), and returns one JSON document. About 3 s of compute, 71 KB gzipped. Nothing has to stay running between requests, which is what serverless hosting needs.
- **API** (`api/index.py`, FastAPI; the Vercel function): `/api/snapshot` with a 30-second in-memory cache and gzip, `/api/health`. Cache-Control lets a CDN reuse a snapshot for 30 s.
- **Front end** (`public/`): plain HTML, CSS and JavaScript, no build step. Charts are hand-drawn SVG. The page polls every 30 s.
- **Local server** (`serve.py`): API plus static files on http://127.0.0.1:8000, optional daily retrain thread.

Deployment on Vercel

- The repository root is the Vercel project (preset "Other", root `./`, no build step). `vercel.json` serves `public/` as static files and rewrites `/api/<route>` to the Python function as `/api/index?route=<route>`.
- **Routing fix.** The first deploy answered 404: after the rewrite the function sees a different path than the browser asked for. The API now dispatches on `?route=` (or the last path segment), so it answers whatever path it receives.
- **No native ML libraries.** The second deploy failed with `libgomp.so.1: cannot open shared object file`: LightGBM and XGBoost need the OpenMP runtime, which Vercel's Python image does not include. The function now predicts with the numpy tree evaluator, so `requirements.txt` holds only numpy, pandas and FastAPI (a much smaller bundle). Training libraries live in `requirements-dev.txt`, used locally and by the daily GitHub Actions job.
- Verified in a clean environment with only the runtime requirements: snapshot built in 2.5 s, no LightGBM, XGBoost or scikit-learn loaded, same probabilities as the native local server.
- **Daily updates reach the site automatically:** the GitHub Actions retrain commits the store and registry, the push redeploys Vercel, and the snapshot reads `active.json` on each request.

Is it a continuous stream?

Yes, as continuous as the public source allows: NOAA updates the GOES-19 feed once a minute, the server re-computes at most every 30 s, and the page asks every 30 s, so a new minute appears within about a minute of NOAA publishing it. The page shows the data age and warns if the feed is stale.

Snapshot contents

Key	Contents
now	state (QUIET / RISE / FLARE / DECAY), flux, class, background and its class, XRS-A, ongoing flare
forecasts.C1, forecasts.M1	probability, alert on/off, threshold and m-of-n rule, climatology, model version and train_end, alerts in 48 h, held-out skill (POD, FAR, CSI, false alarms/day, lead, precursor fraction, ROC AUC)
flares	every nowcast flare in the window with start, peak, end, class, fusion label
series	per-minute time, XRS-B, XRS-A, background, state, P(C1), P(M1) for the charts
learning	recent daily retraining decisions from log.jsonl
meta	source, generated_at, data_end, data_age_min, stale, compute_s

Alarm levels

Level shown	When	Tone
All clear	No flare, neither forecast alarm on	good (green)

Level shown	When	Tone
Flare watch (C)	C1 forecast alarm on	warning (amber)
Major watch (M)	M1 forecast alarm on	serious (orange)
Flare in progress — class shown	Nowcast in FLARE state	serious; critical (red) for M and X
Flare ending	Nowcast in DECAY state	good

Each alarm is logged with its outcome (did a flare follow, and how big). Optional alarm sound (Web Audio) and desktop notifications (Notification API) can be switched on. Preview modes for quiet days: `?demo=watch`, `?demo=major`, `?demo=flare`; theme preview: `?phase=day|dawn|night`.

9. UI / UX design

Design journey

- **Version 1–2:** a functional dashboard with charts and tables, then simplified around one question: "Do I need to worry right now?" — which led to the alarm board.
- **Version 3–4:** redesigned from two reference images supplied by the user (img1, img2: Dribbble shots in a flat, illustrated "deep space" style with plum backgrounds, magenta/orange accents, big rounded type and ring gauges). We did **not** copy the artwork; all illustrations (Sun with a zig-zag corona, orbiting satellite, sparkles, the four how-it-works icons) are original SVG drawn in that style.
- **Themes:** day, dawn/dusk and night palettes chosen from the viewer's local clock.
- **Hero fix:** the giant status title used to be drawn over the lower half of the Sun with very tight letter spacing. The hero is now a stack — Sun on top, status text below — so they never overlap at any width; the title has normal spacing, a smaller maximum size (about 70 px desktop, 56 px phone) and balanced wrapping for longer states such as "Major watch".

Design principles

- **One answer first.** The hero shows a single big status in plain words ("All clear", "Major watch", "M2.4 flare"), with one sentence explaining it.
- **Then evidence.** Right beside it: the X-ray brightness chart and four rings — C chance, M chance, flares caught (66 %), false alarms (36 %).
- **Then detail.** Below: alarm log, chance chart, flares per 6 h, model cards with skill numbers, daily retraining log, and a four-step explainer.
- **Honest about the model.** Every forecast shows its version, training date, alarm rule and held-out skill, including its false-alarm rate.
- **Plain language.** "X-ray brightness", "chance in the next 15 min", "flares caught" instead of jargon; technical terms in tooltips.



Figure 9.1 — Night theme, desktop: stat tiles, the alarm board (illustrated Sun with the status below it), the flux chart, model pill and the four rings.

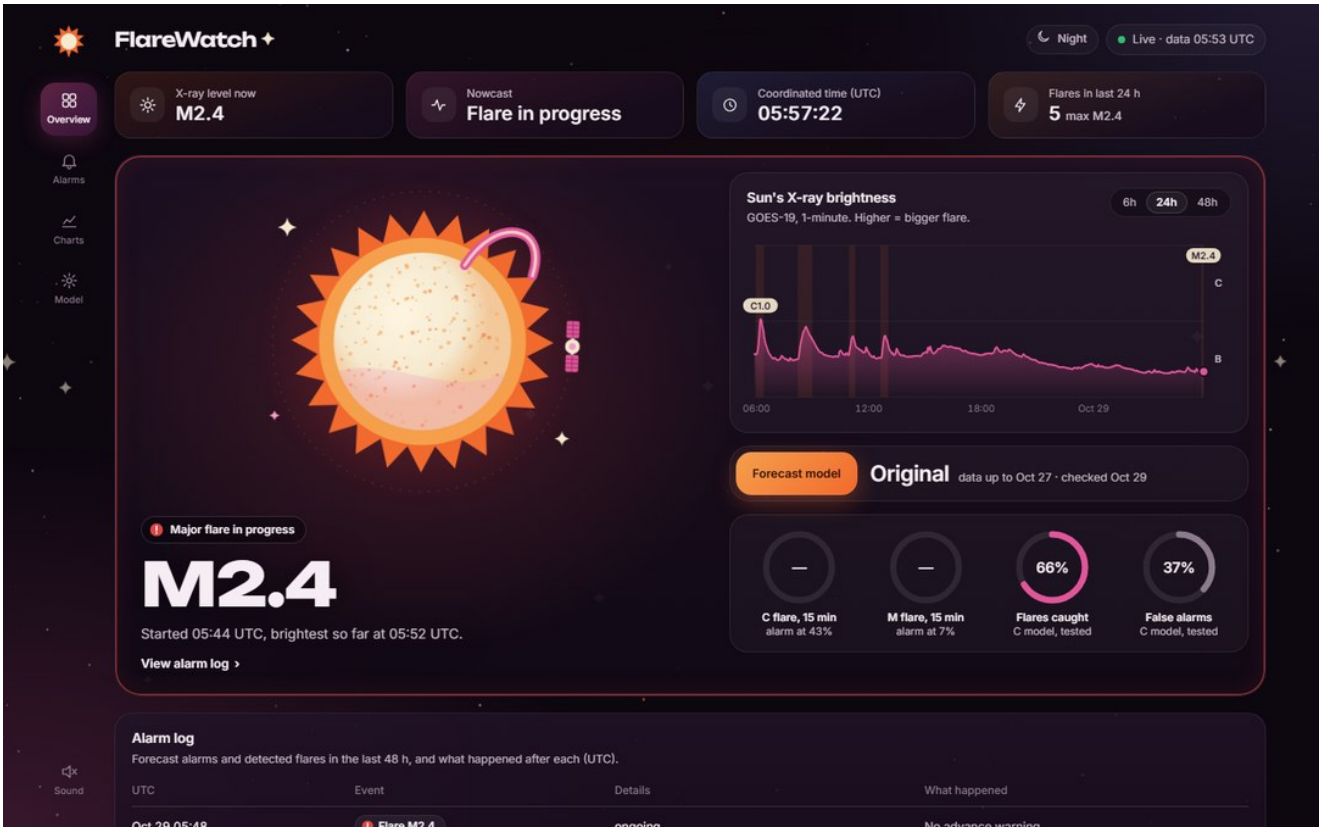


Figure 9.2 — Flare in progress (demo mode): the Sun turns red with a prominence arc, the status shows the flare class, and the tone chip goes critical.

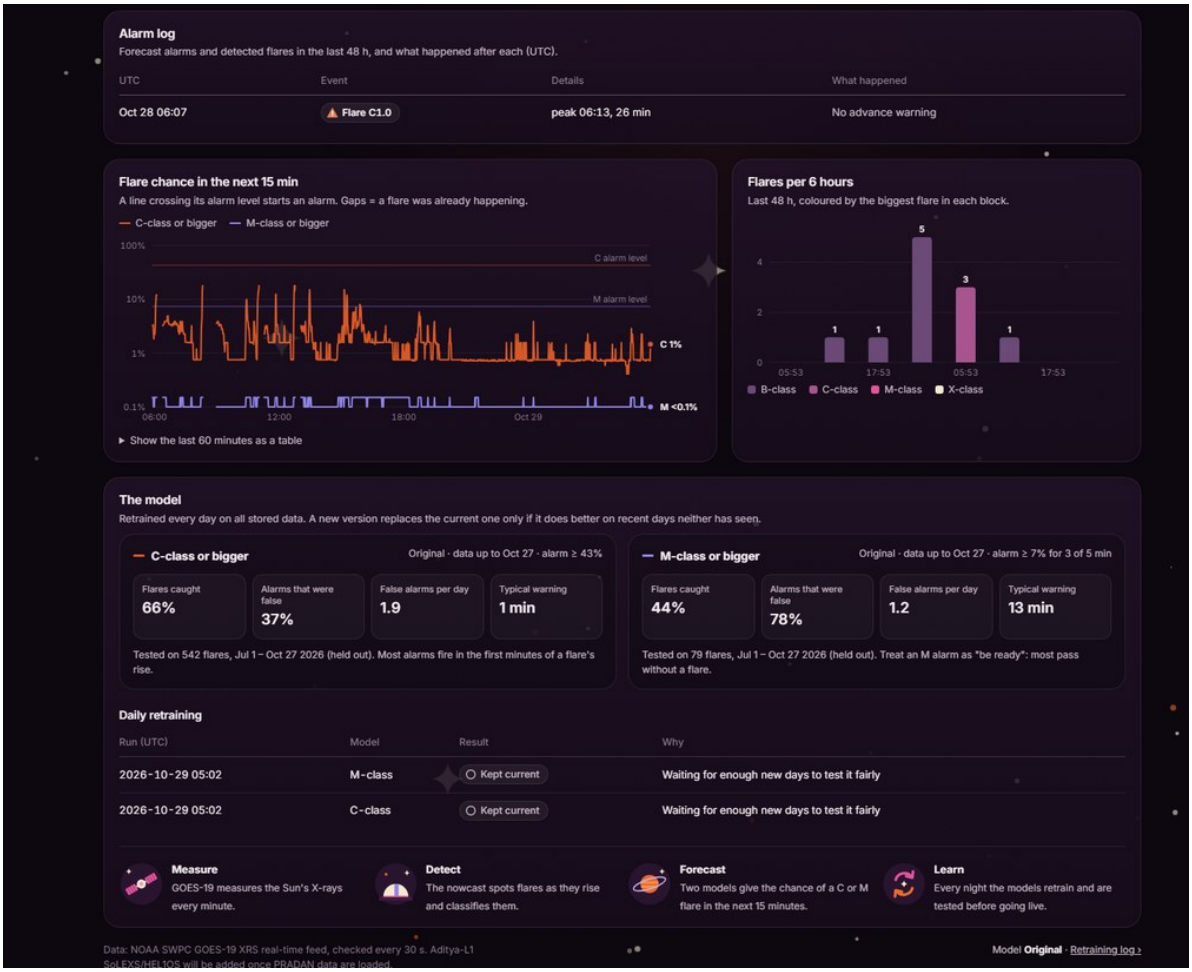


Figure 9.3 — Lower part of the page: alarm log with outcomes, flare-chance chart with alarm levels, flares per 6 h, model cards, retraining log and the how-it-works steps.

Time-of-day themes

Local time	Theme	Look
05:00 – 08:00	Dawn	Twilight sky, rose-to-peach horizon, stars fading upward, dark glass panels
08:00 – 17:00	Day	Light sky with flat clouds, white glass panels, darker chart colours for contrast
17:00 – 20:00	Dusk	Uses the dawn palette
20:00 – 05:00	Night	Deep plum with a full starfield (the original look)

The theme is set by a small script in the page head *before* the first paint (no flash of the wrong theme), re-checked every minute, and can be pinned with the theme button (Auto → Dawn → Day → Night; remembered in the browser). All colours are CSS variables, so each theme is one block of overrides.

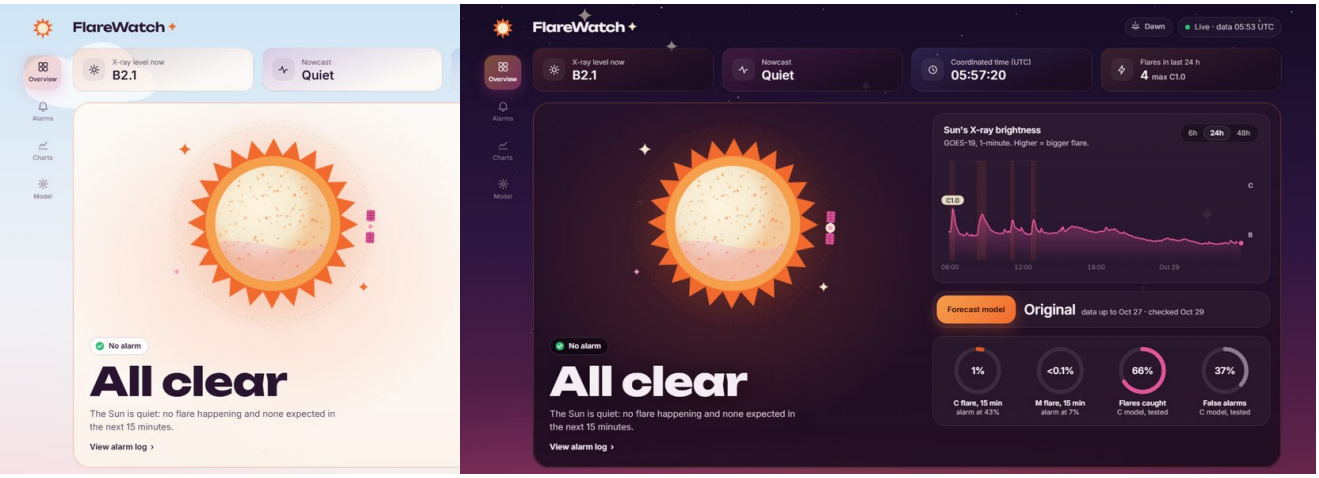
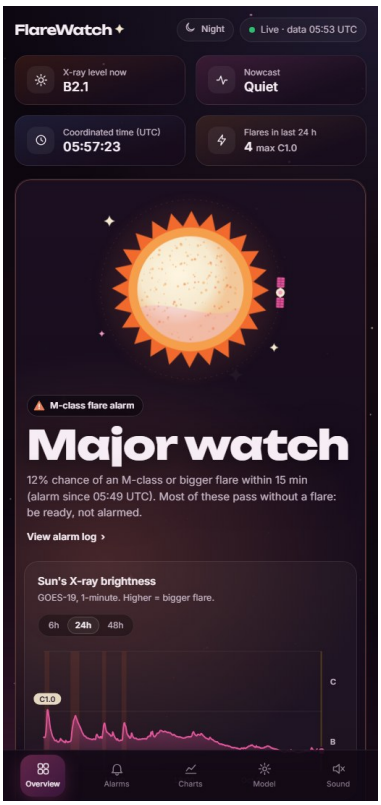


Figure 9.4 — Day (left) and dawn (right) themes of the same page.

Colour and accessibility

Role	Night / dawn	Day	Rule
X-ray flux line	#e2569b	#c8387f	Data colours checked with a palette validator for colour-blind separation and contrast against each theme's surface
C-flare chance	#d95926	#eb6834	
M-flare chance	#9085e9	#6250d6	
Status: good / warning / serious / critical	#2fbf71 / #fab219 / #ec835a / #e04848	same	Reserved for alarm state only, always with a text label and icon

- Status is never shown by colour alone: every tone has a word ("All clear", "Major watch") and the alarm board is an ARIA live region so screen readers announce changes.
- Charts have legends, direct labels, crosshair tooltips, and a "last 60 minutes as a table" view.
- Text uses ink colours, never the series colour; a 2 px surface halo keeps labels readable over lines.
- Phone layout: the icon rail becomes a bottom navigation bar, the Sun shrinks and the text moves below it; grids use minmax(0, 1fr) so nothing overflows.



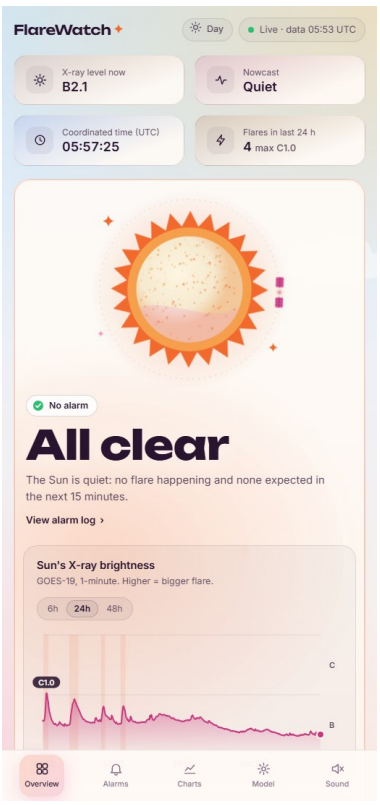


Figure 9.5 — Phone layout: night theme in a Major watch (left) and day theme (right).

10. Testing, code layout and history

Automated tests

Suite	Tests	Covers
tests/test_nowcast.py	19 passed	Background estimator, onset/end rules, re-rise split, class from flux, fusion, gaps, streaming = batch, readers
tests/test_forecast.py	11 passed	Feature causality (no future data), gap handling, labels, model save/load, alert rule, streaming engine, numpy engine = native LightGBM/XGBoost for C1, M1, C1_transfer
tests/test_continual.py	5 passed	Store merge priority and pruning, registry promote/rollback, evaluation window and embargo, gate decisions

All 35 run with one command from the repository root: `python -m pytest -q`.

Repository layout

Everything lives in one repository with one root, so the root is both the Python project and the Vercel site. Nothing imports across sibling folders; the packages sit next to the API that uses them.

```
Aditya-L1-Space-Weather-Monitoring-Pipeline/
├── api/index.py          Vercel serverless function (FastAPI): /api/snapshot, /api/health
├── public/               FlareWatch page: index.html, styles.css, app.js
├── flaremon/             snapshot builder: SWPC feed → nowcast → forecast → JSON
├── l1_nowcast/           background, detector + fusion, data sources, validation
├── l1_forecast/          features, labels, model, walk-forward, alerts, streaming engine
├── continual/            store.py, registry.py, retrain.py; store/ (data), registry/ (models)
├── models/               validated base models C1, M1, C1_transfer
├── notebooks/            nowcast.ipynb, forecast.ipynb, original/ (first prototype)
├── outputs/              nowcast/ and forecast/ tables and figures
├── tests/                35 tests
├── docs/                 component docs + this report
├── serve.py              local server (API + page, optional daily retrain)
├── requirements.txt      runtime only: numpy, pandas, fastapi (what Vercel installs)
├── requirements-dev.txt  notebooks, training, tests
├── vercel.json, .vercelignore
└── .github/workflows/daily-retrain.yml
```

Software

Python with pandas 3.0, NumPy 2.5, LightGBM 4.7, XGBoost 3.4, scikit-learn 1.9 (calibration, metrics), FastAPI 0.162 + Uvicorn, PyArrow (Parquet). The deployed function needs only NumPy, pandas and FastAPI. The front end uses no framework; fonts are Inter and Unbounded from Google Fonts.

Commit history of this work

Commit	What
5cfa3ef	Add streaming nowcast and forecast pipelines (nowcast/, forecast/)
6c9fbbc	Rewrite README around the nowcast/forecast results; remove the duplicate Readme.md (case clash)
e58a431	Add FlareWatch dashboard and daily continual retraining (continual/, workflow)
7c27348	Dashboard: dawn / day / night themes from the viewer's local time
a6fe06a	One-root layout for Vercel (packages, api/, public/ at the root; single requirements; vercel.json) + this report
a874c00	Fix 404 on Vercel: route /api/* by ?route= so the function answers whatever path it sees
ffccbed	Hero: stack the status title below the Sun so it never overlaps it
1b8296a	Fix Vercel libgomp error: numpy tree inference, native ML libraries removed from the runtime; CLAUDE.md
(latest)	This report, updated

Problems met and fixed along the way

- Mobile grid overflow (fixed with minmax(0, 1fr)); hero text overlapping the Sun on phones (text moved below a smaller Sun at ≤ 700 px).
- A hidden banner still showing (added [hidden]{display:none}); panels stretching to the rail height (align-content: start).
- Starfield blotches caused by string coordinates in the sparkle drawing (numbers passed instead).
- Replay test print failed on a window with no positive minutes (BSS undefined) — only the report line was affected.
- Vercel 404 on /api/snapshot (path changed by the rewrite) — fixed by dispatching on ?route=.
- Vercel libgomp.so.1 missing for LightGBM — fixed by numpy inference of the saved trees.
- "All clear" title overlapping the Sun — fixed by stacking the hero.

11. Limitations and next steps

Current limitations

- **Aditya-L1 not yet scored.** The SoLEXS/HEL1OS readers, calibration, channel settings and transfer model are implemented and unit-tested, but real Aditya-L1 files have not been run because they need the user's PRADAN login.
- **Forecast warning is short for C flares.** Most C1 "warnings" come in the first minutes of the rise. Only M1 shows a real precursor.
- **M1 false alarms.** FAR 0.78 — useful as a risk level, not as a hard trigger.
- **Skill panel numbers** on the dashboard still come from the original Jul–Oct test of the base model, not from the daily-retrained versions.
- **Single public source.** If SWPC is down, the dashboard shows a stale-data warning; there is no fallback feed yet.

Next steps

#	Step	Detail
1	Load Aditya-L1 data from PRADAN	Download SoLEXS/HEL1OS files; tune the nowcast with tune_channel; build an Aditya-L1 flare catalogue; compare with GOES; then retrain the forecast on Aditya-L1's own catalogue with walk_forward. Use DirectoryWatchSource for near-real-time files.
2	Watch the Vercel deployment	Deployed. Confirm the page loads data after the libgomp fix, check function duration (about 3 s per refresh, cached 30 s) and that the daily GitHub Actions commit triggers a redeploy that serves newly promoted models.
3	First fair retrain comparison	Around 1 Nov 2026 the challenger and champion will have ≥ 3 unseen days; watch log.jsonl and the dashboard's retraining log.
4	Live skill panel	Compute rolling 30-day POD/FAR/CSI for the active version from stored data and show it next to the held-out skill.
5	Push notifications	Web Push / email / Telegram alerts for Major watch and M/X flares, so users need not keep the page open.
6	Better M-flare lead	Add longer-horizon targets (30–60 min), magnetogram or active-region features (e.g. SDO/HMI SHARP), and probabilistic "risk band" display.
7	Robustness	Fallback data source (GOES secondary satellite), monitoring of the daily job, and data-drift checks before promotion.

Glossary of scores

Term	Meaning
POD (recall, hit rate)	Share of real flares that were caught. 0.66 = 66 of every 100.
FAR (false-alarm ratio)	Share of alarms that were not followed by a flare. Lower is better.
Precision	1 − FAR: share of alarms or detections that were real.
CSI (critical success index)	hits / (hits + misses + false alarms). Rewards catching flares without crying wolf; 1 is perfect.
F1	Harmonic mean of precision and recall.
ROC AUC	How well the probability ranks flare minutes above quiet minutes; 0.5 is random, 1 is perfect.
BSS (Brier skill score)	Probability accuracy relative to always forecasting the climatological rate; above 0 means better than climatology.
Log loss	Penalty for confident wrong probabilities; used by the retraining gate.
Lead time / warning	Minutes between the forecast alarm and the nowcast alarm (or NOAA start) of the flare it predicted.

Term	Meaning
Embargo	A gap between training and test data so that slowly-changing conditions do not leak across the boundary.
Walk-forward	Train on the past, test on the next month, move forward, repeat — the way the model would run live.
GOES class	Peak 1–8 Å flux: A, B, C (10^{-6}), M (10^{-5}), X (10^{-4} W/m²); the number is the multiplier, e.g. M2.4 = 2.4×10^{-5}.